SIMATS ENGINEERING

ASSESSMENT TOOL 2

Q1. For the grammar E → E + T | T, remove left recursion.

Given: E → E + T | T

This is left recursive because E → E + T.

The standard elimination form is:

E → T E'
E' → + T E' | ε

Answer: E → T E' and E' → + T E' | ε

Q2. Find FIRST(E) for: E → T E', E' → + T E' | ε

Given: E → T E' and E' → + T E' | ε

Since E starts with T:

FIRST(E) = FIRST(T)

Assuming T → id:

FIRST(T) = { id }

Answer: FIRST(E) = { id }

Q3. For the grammar E → T + E | T, check if it is ambiguous or not.

Grammar: E → T + E | T

For a string such as T + T + T, there is only one possible structure:

T + (T + T)

The grammar produces only one parse tree for each valid string.

Answer: The grammar is unambiguous.

Q4. Identify whether the string aba is valid for: S → aSa | b

Grammar: S → aSa | b

Derivation:

S → aSa → aba

Therefore, aba is generated by the grammar.

Answer: Yes, aba is valid.

Q5. Check whether the string ab is generated by: S → aA, A → b

Grammar: S → aA, A → b

Derivation:

S → aA → ab

Answer: Yes, ab is generated.

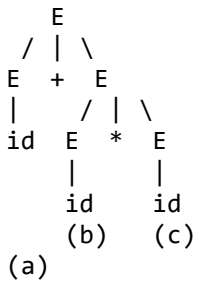
Q6. Construct a parse tree for a + b * c using: E → E + E | E * E | id

Grammar: E → E + E | E * E | id

For normal arithmetic precedence, * has higher precedence than +, so the expression is grouped as:

a + (b * c)

Parse tree structure:



Answer: Result: a + (b * c)

Q7. Apply left factoring for: S → aA | aB | b

Given: S → aA | aB | b

The common prefix is 'a'. Factoring it out:

```
S  → aS' | b
S' → A | B
```

Answer: S → aS' | b and S' → A | B

Q8. Find FOLLOW(A) for: S → A B, B → b

Given: S → AB, B → b

Since A is followed by B:

FOLLOW(A) = FIRST(B)

Since B → b, FIRST(B) = { b }

Answer: FOLLOW(A) = { b }

Q9. Given the grammar S → aSb | ε, check whether the string aaabbb is valid using derivation.

Grammar: S → aSb | ε

Derivation:

S → aSb → aaSbb → aaaSbbb → aaabbb

Therefore, aaabbb is generated.

Answer: Yes, aaabbb is valid.

Q10. Convert the following ambiguous grammar into an unambiguous grammar: E → E + E | E * E | id

Given: E → E + E | E * E | id

This grammar is ambiguous because precedence and associativity are not specified. An unambiguous version enforcing precedence (* higher than +) and left associativity is:

```
E → E + T | T
T → T * F | F
F → id
```

Answer: E → E + T | T , T → T * F | F , F → id

Q11. Identify whether the grammar S → S a | b is suitable for top-down parsing.

Given: S → Sa | b

The grammar has left recursion (S → Sa). Top-down parsers such as recursive descent parsers cannot directly handle left-recursive grammars.

Removing left recursion gives:

```
S  → bS'
S' → aS' | ε
```

Answer: The given grammar is not directly suitable for top-down parsing because it contains left recursion.

Q12. Write a recursive descent parser for the grammar: S → aS | b

C program implementing the recursive descent parser:

```
#include <stdio.h>
#include <string.h>

char input[100];
int pos = 0;

void S()
{
    if (input[pos] == 'a')
    {
        pos++;
        S();
    }
    else if (input[pos] == 'b')
    {
        pos++;
    }
    else
    {
        printf("Error");
    }
}

int main()
{
    printf("Enter string: ");
    scanf("%s", input);
    S();
    if (input[pos] == '\0')
        printf("Valid string");
    else
        printf("Invalid string");
    return 0;
}
```

Q13. Construct the operator precedence table for: E → E + E | E * E | id

Grammar: E → E + E | E * E | id. Operators: +, *, id, $. Since * has higher precedence than +, the operator precedence table is as follows:

	+	*	id	$
+	>	<	<	>
*	>	>	<	>
id	>	>	-	>
$	<	<	<	=

Q14. What is panic mode error recovery?

Panic mode error recovery is a simple error recovery technique used by parsers. When the parser finds an error, it:

1. Discards input symbols.

2. Continues discarding until it finds a synchronizing symbol.

3. Common synchronizing symbols are ; , } , or $.

4. Parsing then continues from that point.

Example: int a = ; int b = 10;

The parser detects an error at '=' because an expression is missing. It skips input until a suitable symbol such as ';' is found and then continues parsing.

Answer: Panic mode recovery skips erroneous input until a synchronizing token is found, allowing parsing to continue.

Q15. Why is error recovery important in parsing?

Error recovery is important because it allows the compiler to continue parsing after finding an error instead of stopping immediately.

Main advantages:

1. Detects multiple errors in one compilation.

2. Prevents the compiler from stopping at the first error.

3. Gives better error messages.

4. Saves time during debugging.

5. Helps the compiler continue processing the remaining program**Answer:** Error recovery improves the compiler's ability to detect and report multiple errors efficiently while continuing the compilation process.